

Multi-Task ModernTCN Perception-Enhanced LPV-MPC for Path Tracking of a Diagonal Dual-Steer AGV

AUTHOR NAME¹

¹Affiliation placeholder.

Corresponding author: Author Name (e-mail: email@example.com).

ABSTRACT

This paper proposes a multi-task ModernTCN perception-enhanced linear parameter-varying model predictive control framework for path tracking of a diagonal dual-steer automated guided vehicle under sloped, transitional, and disturbed operating conditions. In conventional LPV-MPC, inaccurate scheduling variables, especially slope-related information, may cause model mismatch and degrade closed-loop tracking performance. To address this issue, a lightweight temporal perception module is developed to infer the operating condition, steering direction, and slope-related scheduling variable from a short history of proprioceptive measurements. The estimated information is then incorporated into the online scheduling process of the LPV-MPC controller while retaining the constraint-handling capability and interpretability of the model-based controller. The proposed method is evaluated on a unified dataset and a unified MATLAB/Simulink LPV-MPC closed-loop platform against GRU, conventional TCN, zero-slope LPV-MPC, IMU-based LPV-MPC, and a true-slope oracle baseline. The experiments include offline perception evaluation, closed-loop baseline comparison, multi-route benchmarking, robustness tests under measurement noise and plant parameter perturbations, a causal ModernTCN ablation, and real-time computational assessment. The results show that ModernTCN provides the most robust overall closed-loop performance among the learning-based methods, substantially improves tracking quality over non-AI scheduling baselines, and maintains its advantage across multiple routes and disturbance levels. The timing benchmark further indicates that the core ONNXRuntime inference plus MPC solving pipeline has sufficient computational margin for a 10 ms control cycle. These findings suggest that multi-task temporal perception is an effective scheduling enhancement for LPV-MPC, while also highlighting the necessity of closed-loop validation beyond offline perception metrics.

INDEX TERMS

Automated guided vehicle, closed-loop validation, diagonal dual-steer vehicle, linear parameter-varying model predictive control, ModernTCN, multi-task learning, path tracking, slope estimation, temporal perception.

I. INTRODUCTION

AUTOMATED guided vehicles (AGVs) are increasingly used in factory logistics, warehouse transportation, and flexible manufacturing systems. In these applications, the vehicle is required to track reference paths accurately while operating under constrained actuation, varying loads, turning transitions, and uneven or sloped ground conditions. For diagonal dual-steer AGVs, the path-tracking problem is further complicated by the coupled longitudinal, yaw, steering, and load-transfer dynamics caused by the active drive-steer wheels and passive support wheels. Therefore, a control

method for such a platform should not only reduce lateral and heading errors but also maintain smooth control actions and constraint satisfaction under changing operating conditions.

Model predictive control (MPC) is a natural candidate for constrained vehicle path tracking because it can explicitly account for future reference information, actuator limits, and multi-variable system interactions [?]. For nonlinear vehicle systems, linear parameter-varying MPC (LPV-MPC) provides a practical compromise between model fidelity and online computational tractability by updating the prediction model according to scheduling variables such as velocity, yaw

rate, and road slope [?]. However, the effectiveness of LPV-MPC depends strongly on whether the scheduling variables accurately represent the current operating condition. If the slope or condition-related scheduling information is unavailable, biased, or delayed, the controller may operate with an inconsistent prediction model and generate degraded closed-loop behavior.

In the considered AGV system, the ground slope is not merely a diagnostic signal; it directly influences the longitudinal dynamics, load transfer, and the scheduling of the LPV model. A nominal zero-slope scheduler or a simplified IMU-based scheduler can therefore be insufficient on complex sloped factory routes. This motivates the use of a learned temporal perception module that estimates scheduling-relevant information from the recent history of proprioceptive measurements. Instead of replacing the model-based controller or learning the control action end-to-end, the learning module is designed to provide interpretable intermediate estimates to the LPV-MPC controller.

This work proposes a multi-task ModernTCN perception-enhanced LPV-MPC framework for diagonal dual-steer AGV path tracking. The perception module takes a 128-step window of 19 measured or derived features and jointly predicts three outputs: the main operating condition, the steering direction, and a slope-related scheduling variable. The resulting estimates are used to enhance the online LPV-MPC scheduling process. The controller remains model-based and constrained, whereas the neural module serves as a scheduling perception layer.

The main contributions of this paper are summarized as follows:

- A multi-task temporal perception-enhanced LPV-MPC framework is proposed for path tracking of a diagonal dual-steer AGV under sloped, stall-like, and turning-transition operating conditions.
- A unified multi-sensor time-window dataset and leakage-free training protocol are constructed for operating-condition classification, steering-direction recognition, and slope scheduling estimation.
- A lightweight ModernTCN-based perception module is integrated into MATLAB/Simulink LPV-MPC through ONNX, and it is compared with GRU and conventional TCN baselines under the same dataset, controller, and closed-loop platform.
- Extensive closed-loop evaluations are conducted against zero-slope LPV-MPC, IMU-based LPV-MPC, a true-slope oracle LPV-MPC, GRU, TCN, and ModernTCN under single-route, multi-route, disturbance, and real-time settings.
- A causal ModernTCN ablation is included to show that strong offline perception metrics do not necessarily imply robust closed-loop performance, highlighting the need for closed-loop validation of learning-enhanced MPC systems.

The remainder of this paper is organized as follows. Section II reviews related work on MPC-based path track-

ing, learning-enhanced MPC, and temporal neural networks. Section III introduces the diagonal dual-steer AGV model and LPV-MPC formulation. Section IV describes the multi-task perception problem and dataset construction. Section V presents the ModernTCN-based scheduling perception module. Section VI defines the experimental protocol. Section VII reports and discusses the closed-loop, robustness, ablation, and real-time results. Section VIII concludes the paper.

II. RELATED WORK

A. MPC AND LPV-MPC FOR VEHICLE AND AGV PATH TRACKING

MPC has been widely used for automated vehicle and mobile robot path tracking because it can explicitly handle multi-variable dynamics, future trajectory references, actuator saturation, and input-rate constraints [?]. For nonlinear vehicle systems, nonlinear MPC can provide high prediction fidelity, but its computational demand can be prohibitive for fast online implementation. LPV-MPC addresses this tradeoff by constructing a family of local linear models parameterized by measurable or estimable scheduling variables [?]. This structure is attractive for AGV and autonomous vehicle path tracking, where the dynamics vary with velocity, yaw rate, road slope, and tire-force operating conditions.

Existing LPV-MPC studies often assume that the key scheduling variables can be measured directly or estimated reliably from simple physical sensors. However, in industrial AGV operation, slope changes, turning transitions, load transfer, actuator interaction, and stall-like disturbances can make the scheduling information uncertain or biased. This paper focuses on this scheduling mismatch problem and investigates whether multi-task temporal perception can provide more effective scheduling information for LPV-MPC.

B. LEARNING-ENHANCED MPC

Learning-enhanced MPC combines data-driven models with constrained optimal control. Existing studies can be broadly categorized into learning prediction models, learning model residuals, learning uncertainty descriptions, adapting cost or constraint parameters, or accelerating online optimization [?]. These approaches have demonstrated that learning components can improve model accuracy or computational efficiency while retaining some advantages of MPC, such as constraint handling and predictive decision making.

Most learning-enhanced MPC studies for vehicles focus on learning the vehicle dynamics, compensating modeling residuals, or replacing part of the optimizer. By contrast, the present work does not directly replace the physical model or the MPC optimizer. Instead, it learns interpretable scheduling-related perception variables, including the operating condition, steering direction, and slope-related scheduling estimate. This positioning keeps the control action generated by the MPC while allowing the controller to use richer temporal information than a single instantaneous sensor measurement.

C. TEMPORAL NEURAL NETWORKS FOR MULTI-VARIABLE SEQUENCE PERCEPTION

Recurrent neural networks, such as GRU, have been widely used for sequence modeling because their gating mechanism can encode temporal dependencies in compact hidden states [?]. Temporal convolutional networks (TCNs) provide another strong sequence modeling baseline by using one-dimensional convolutions and residual structures, and they have shown competitive performance against recurrent networks in generic sequence modeling tasks [?]. ModernTCN further revisits pure convolutional structures for general time-series analysis by combining large-kernel temporal convolutions and modern channel-mixing designs [?].

Although TCN and ModernTCN have shown strong performance in time-series modeling, their use as deployable perception modules inside constrained MPC loops remains insufficiently explored. In a closed-loop control system, prediction errors may change the future state distribution through feedback, and offline accuracy alone may not predict closed-loop robustness. This paper therefore evaluates the temporal models not only by offline perception metrics but also by closed-loop tracking, control smoothness, constraint behavior, robustness, and real-time feasibility.

D. RESEARCH GAP AND POSITIONING OF THIS WORK

The above review indicates that LPV-MPC is suitable for constrained vehicle path tracking, learning-enhanced MPC is a promising way to improve model-based control, and temporal neural networks can extract useful information from multi-sensor sequences. However, there remains a gap in learning interpretable multi-task scheduling perception for LPV-MPC and validating it through systematic closed-loop experiments. This work addresses the gap by integrating a lightweight ModernTCN perception module into an LPV-MPC-controlled diagonal dual-steer AGV and comparing it against non-AI, sensor-based, oracle, recurrent, convolutional, and causal-ablation baselines.

III. DIAGONAL DUAL-STEER AGV MODELING AND LPV-MPC FORMULATION

A. AGV CONFIGURATION AND COORDINATE DEFINITION

The considered platform is a diagonal dual-steer drive AGV. The left-front and right-rear wheels are active drive-steer wheels, whereas the right-front and left-rear wheels are passive support wheels. The vehicle motion is described in both the global frame and the body-fixed frame. The path-tracking objective is to regulate the lateral error, heading error, velocity error, and yaw-rate-related tracking error while respecting the input and input-rate constraints.

The discrete vehicle state used in the simulation platform is

$$x = [X \ Y \ \psi \ v \ \omega \ \delta_{lf} \ \delta_{rr} \ \beta]^T, \quad (1)$$

where X and Y are the global positions, ψ is the heading angle, v is the longitudinal velocity, ω is the yaw rate, δ_{lf}

fig1_agv_configuration.pdf

FIGURE 1. Configuration and motion variables of the diagonal dual-steer AGV. The LF and RR wheels are active drive-steer wheels, whereas the RF and LR wheels are passive support wheels. The variables v , β , ψ , ω , R , L , W , δ_{lf} , δ_{rr} , and θ denote the longitudinal velocity, sideslip angle, heading angle, yaw rate, turning radius, vehicle length, track width, front-left steering angle, rear-right steering angle, and road slope, respectively. The arrows $F_{x,lf}$ and $F_{x,rr}$ indicate the driving forces applied at the active wheels. The steering angles δ_{lf} and δ_{rr} are steering actuator states, while the MPC control input is $u = [F_{cmd} \ \omega_{cmd}]^T$.

and δ_{rr} are the steering angles of the left-front and right-rear wheels, and β is the sideslip angle. The control input is

$$u = [F_{cmd} \ \omega_{cmd}]^T, \quad (2)$$

where F_{cmd} is the driving force command and ω_{cmd} is the yaw-rate command. The ground slope angle θ is treated as a disturbance and a scheduling-related variable.

B. NONLINEAR STATE-SPACE MODEL

The nonlinear discrete-time vehicle model is written in the compact form

$$x_{k+1} = f(x_k, u_k, \theta_k, p), \quad (3)$$

where p denotes the vehicle and environment parameters. The implementation includes load transfer, tire lateral force limitation, drive-force distribution between the active wheels, steering actuator dynamics, rolling resistance, aerodynamic drag, and slope-induced resistance. The numerical state propagation is performed using a fourth-order Runge–Kutta integration scheme in the simulation model.

The output vector contains the basic vehicle states, IMU-related measurements, motor current estimates, wheel speeds, ground slope, load-transfer-related quantities, tire-utilization-related quantities, and diagnostic variables. These outputs are used both for closed-loop control diagnostics and for constructing the temporal perception features.

C. LPV REPRESENTATION AND SCHEDULING VARIABLES

To obtain an online tractable predictive controller, the non-linear AGV model is represented by an LPV model over a scheduling vector. The local prediction model can be expressed as

$$x_{k+1} = A(\rho_k)x_k + B(\rho_k)u_k + E(\rho_k)d_k, \quad (4)$$

$$y_k = C(\rho_k)x_k + D(\rho_k)u_k, \quad (5)$$

where ρ_k is the scheduling vector, d_k denotes measured or estimated disturbances, and y_k is the controlled output vector. In this work, the scheduling vector is defined as

$$\rho_k = [v_k \quad \omega_k \quad \theta_k]^T. \quad (6)$$

The slope-related component can be obtained from different sources depending on the controller variant: zero-slope scheduling, IMU-based scheduling, true-slope oracle scheduling, or learned ModernTCN scheduling.

D. MPC OPTIMIZATION PROBLEM

At each sampling instant, the LPV-MPC controller updates the prediction model using the current scheduling vector and solves a constrained finite-horizon optimization problem. A generic form of the cost function is

$$J_k = \sum_{i=1}^{N_p} |y_{k+i|k} - y_{k+i}^{\text{ref}}|_Q^2 + \sum_{i=0}^{N_c-1} |u_{k+i|k}|_R^2 + \sum_{i=0}^{N_c-1} |\Delta u_{k+i|k}|_{R_\Delta}^2, \quad (7)$$

where N_p and N_c are the prediction and control horizons, respectively; Q , R , and R_Δ are weighting matrices; and $\Delta u_{k+i|k} = u_{k+i|k} - u_{k+i-1|k}$. The optimization is subject to the LPV prediction model and input constraints,

$$u_{\min} \leq u_{k+i|k} \leq u_{\max}, \quad (8)$$

$$\Delta u_{\min} \leq \Delta u_{k+i|k} \leq \Delta u_{\max}. \quad (9)$$

Additional state or output constraints can be included according to the controller implementation.

E. INTEGRATION OF LEARNED SCHEDULING ESTIMATES

The proposed learning module does not directly output the control command. Instead, it estimates scheduling-relevant variables and passes them to the LPV-MPC update layer. The learned scheduling estimate is denoted by

$$\hat{\theta}_k = g_\phi(Z_{k-L+1:k}), \quad (10)$$

where $g_\phi(\cdot)$ is the trained ModernTCN perception model, $Z_{k-L+1:k}$ is the historical input window, and L is the sequence length. The scheduled slope used by the controller is conditioned by deployment-side safeguards,

TABLE 1. Dataset Contract Used by All Learning-Based Methods

Item	Value
Vehicle type	Diagonal dual-steer drive AGV
Active drive-steer wheels	LF, RR
Passive support wheels	RF, LR
Sampling time	0.01 s
Sequence length	128 steps
Input dimension	19
Label time policy	Current window end
Prediction horizon for labels	0 step
Split policy	Run-level no-window leakage
Scaler policy	Fit on training set only and apply to validation, test, and online inference
Training windows	18,302
Validation windows	2,607
Test windows	3,733

$$\theta_k^{\text{sch}} = \mathcal{S}(\hat{\theta}_k; \theta_{\max}, \dot{\theta}_{\max}, \theta_{\text{dz}}), \quad (11)$$

where $\mathcal{S}(\cdot)$ denotes the dead-zone, magnitude-limiting, and rate-limiting operation. In the current deployment configuration, the slope magnitude limit, slope rate limit, and MPC dead-zone are set to 12° , 5° , and 2° , respectively.

IV. MULTI-TASK PERCEPTION PROBLEM AND DATASET CONSTRUCTION

A. MULTI-SENSOR INPUT WINDOW

The perception module uses a fixed-length historical window of proprioceptive signals:

$$Z_k = [z_{k-L+1} \quad z_{k-L+2} \quad \cdots \quad z_k]^T \in \mathbb{R}^{L \times F}, \quad (12)$$

where $L = 128$ and $F = 19$. With a sampling time of $T_s = 0.01$ s, each input window covers 1.28 s of recent vehicle motion. The 19 input features are

$$\mathcal{F} = \{a_x, r_z, I_{lf}, I_{rr}, \omega_{w,lf}, \omega_{w,rr}, \delta_{lf}, \delta_{rr}, r_y, \hat{v}, \hat{\dot{v}}, \eta_{ws}, I_\Sigma, I_\Delta^s, |I_\Delta|, a_{x,lp}, \kappa_{\text{proxy}}, a_x/I_\Sigma, \hat{\theta}_{\text{pitch}}\}. \quad (13)$$

The feature names in (13) correspond to the implementation-level variables `accel_x`, `gyro_z`, `I_lf`, `I_rr`, `omega_wheel_l`, `omega_wheel_rr`, `delta_lf`, `delta_rr`, `gyro_y`, `v_hat`, `dv_hat_dt`, `ws_imbalance`, `I_sum`, `I_diff_signed`, `I_diff_abs`, `accel_x_lp`, `kappa_proxy`, `accel_per_curr` and `pitch_angle_est`.

B. TASK AND LABEL DEFINITION

The perception model is trained with three supervised tasks. The first task is the main operating-condition classification,

$$c_k^{\text{main}} \in \{\text{flat}, \text{stall}, \text{slope}\}. \quad (14)$$

The second task is the steering-direction classification,

$$c_k^{\text{turn}} \in \{\text{right}, \text{straight}, \text{left}\}. \quad (15)$$

The third task is the slope-related regression,

TABLE 2. Learning-Based Model Configurations

Model	Main configuration	Role in this paper
ModernTCN	Seed 21, 64 channels, 5 blocks, kernel size 31, same temporal padding	Proposed scheduling perception module
GRU	Seed 101, hidden size 96, 2 layers, input-statistics readout	Recurrent sequence baseline
TCN	Seed 21, 6 blocks, 96 filters, kernel size 3	Conventional convolutional sequence baseline
Causal ModernTCN	Seed 11 selected from causal offline runs	Ablation model, not the default deployment model

$$\theta_k \in \mathbb{R}. \quad (16)$$

These tasks are jointly learned because they describe complementary aspects of the same closed-loop operating condition. The main condition helps distinguish flat, stall-like, and sloped regimes; the steering task captures turning transitions; and the slope regression provides the scheduling quantity used by LPV-MPC.

C. DATA GENERATION AND CONDITION COVERAGE

The dataset is generated from the AGV simulation platform. The data cover flat-road operation, slope operation, stall-like or abnormal resistance conditions, and turning transitions. All learning-based methods in this paper use the same dataset, feature set, label definition, split policy, and normalization policy. This avoids evaluating the neural models under different data conditions.

D. RUN-LEVEL SPLIT AND NORMALIZATION

A run-level split is used to prevent adjacent windows from the same simulation run from appearing in both training and test sets. This is important because heavily overlapping time windows may otherwise leak trajectory-specific information into the test set. The scaler is fitted using the training set only and then applied to the validation set, test set, and online inference pipeline. This protocol is intended to make the offline evaluation closer to the closed-loop deployment setting.

E. BASELINE TRAINING PROTOCOL

The learning-based baselines include GRU, conventional TCN, ModernTCN, and the causal ModernTCN ablation. The default ModernTCN uses seed 21, 64 channels, 5 blocks, kernel size 31, dropout 0.15, expansion ratio 2, and same temporal padding. The GRU baseline uses two recurrent layers with hidden size 96. The conventional TCN baseline uses 6 blocks, 96 filters, and kernel size 3. These configurations are summarized in Table 2.

V. MODERNTCN-BASED MULTI-TASK SCHEDULING PERCEPTION

fig2_moderntcn_architecture.pdf

FIGURE 2. Architecture of the multi-task ModernTCN scheduling perception module. The input window $Z_k \in \mathbb{R}^{128 \times 19}$ is normalized and passed through the ModernTCN-small encoder (stem, 5 large-kernel depthwise temporal-convolution blocks with pointwise channel mixing and residual connections). The shared representation is formed by concatenating the last-step, mean, and max-pooling readouts with input-window statistics. Three task heads produce the main-condition label l_{main} , the steering-direction label l_{turn} , the confidence c , and the slope regression θ . The slope output is conditioned by the deployment guard (gain, magnitude limit, rate limit, dead-zone) and fed into the LPV-MPC scheduling vector $\rho = [v, \omega, \theta_{\text{sch}}]$. The classification outputs l_{main} , l_{turn} , and c further shape the MPC constraints and weights through the scheduling update layer.

A. OVERALL ARCHITECTURE

The proposed perception module is a lightweight multi-task ModernTCN variant. Its input is the normalized time window $Z_k \in \mathbb{R}^{128 \times 19}$. The network extracts temporal and cross-variable representations and produces three outputs:

$$[\hat{p}_k^{\text{main}}, \hat{p}_k^{\text{turn}}, \hat{\theta}_k] = g_\phi(Z_k), \quad (17)$$

where $\hat{p}_k^{\text{main}}$ and $\hat{p}_k^{\text{turn}}$ are classification probabilities, and $\hat{\theta}_k$ is the slope-related regression output.

B. TEMPORAL CONVOLUTION BLOCK

The temporal feature extractor follows the design philosophy of ModernTCN, using large-kernel temporal convolution to capture history-dependent patterns and pointwise channel mixing to capture cross-variable interactions [?]. A generic block can be represented as

$$H^{(\ell+1)} = H^{(\ell)} + \mathcal{M}^{(\ell)} \left(\mathcal{T}^{(\ell)}(H^{(\ell)}) \right), \quad (18)$$

where $\mathcal{T}^{(\ell)}(\cdot)$ denotes temporal convolution and $\mathcal{M}^{(\ell)}(\cdot)$ denotes pointwise channel mixing and nonlinear transformation. The residual connection stabilizes the stacked temporal representation.

C. MULTI-VARIABLE REPRESENTATION AND READOUT

The readout layer combines the last-step representation, temporal statistics, and input-window statistics. This design is motivated by the closed-loop scheduling task: the most recent signal is important for online scheduling, whereas mean and extreme statistics over the window can reveal persistent slope, current imbalance, or turning-transition patterns. The resulting shared representation is used by all three task heads.

D. MULTI-TASK HEADS AND LOSS FUNCTION

The training objective is a weighted multi-task loss,

$$\mathcal{L} = \lambda_{\text{main}} \mathcal{L}_{\text{CE}}(c^{\text{main}}, \hat{p}^{\text{main}}) + \lambda_{\text{turn}} \mathcal{L}_{\text{CE}}(c^{\text{turn}}, \hat{p}^{\text{turn}}) + \lambda_{\theta} \mathcal{L}_{\theta}(\theta, \hat{\theta}), \quad (19)$$

where $\mathcal{L}_{\text{CE}}$ is the classification loss and $\mathcal{L}_{\theta}$ is the slope-regression loss. Additional task-specific weighting terms can be included to emphasize transition windows, flat-road slope suppression, or active-slope regions. The exact hyperparameters used in the current repository are reported in the training configuration files and training reports.

E. ONNX DEPLOYMENT AND SIMULINK INTERFACE

After training in Python, the ModernTCN model is exported to ONNX and loaded by the MATLAB/Simulink closed-loop simulation platform. During online inference, the latest 128-step feature window is normalized using the training-set scaler, passed through the ONNX model, and converted into the operating-condition label, steering-direction label, and slope scheduling estimate. The slope estimate is then conditioned by magnitude, rate, and dead-zone limits before being used by the LPV-MPC update.

F. SCHEDULING SAFEGUARDS

The neural network output is not directly applied as a control input. The final control command is still generated by the constrained LPV-MPC controller. The deployment-side safeguards include slope-output gain, absolute slope limit, slope-rate limit, and MPC dead-zone. These mechanisms reduce the risk of abrupt scheduling jumps. A confidence-triggered fallback strategy is not currently reported as a completed experiment in the public repository; it should therefore be treated as future work rather than a demonstrated result.

G. CAUSAL VARIANT FOR ABLATION

A causal ModernTCN variant is included as an ablation model. Its purpose is to examine whether imposing causal temporal padding in the network improves closed-loop robustness. It is not the default deployed model. The repository results show that the causal model can achieve offline metrics close to the default ModernTCN, but its closed-loop behavior is significantly worse on the main showcase path. Therefore, causal ModernTCN is used in this paper to analyze the mismatch between offline perception metrics and closed-loop control performance.

TABLE 3. Baseline Definitions

Category	Method	Purpose
Non-AI baseline	LPV-MPC theta0	Tests nominal zero-slope scheduling
Sensor baseline	LPV-MPC IMU theta	Tests simplified sensor-based slope scheduling
Oracle baseline	LPV-MPC oracle theta	Provides true-slope scheduling upper-bound reference
Learning baselines	GRU, TCN	Compares recurrent and conventional convolutional sequence models
Proposed method	ModernTCN	Tests multi-task ModernTCN scheduling perception
Ablation	Causal ModernTCN	Tests causal temporal padding and offline-closed-loop mismatch

VI. EXPERIMENTAL PROTOCOL

A. RESEARCH QUESTIONS

The experiments are designed to answer the following research questions:

- RQ1: Does learned temporal perception improve LPV-MPC over zero-slope and IMU-based scheduling?
- RQ2: Does ModernTCN provide better closed-loop performance than GRU and conventional TCN under the same controller and dataset?
- RQ3: How close is ModernTCN to the true-slope oracle LPV-MPC upper-bound reference?
- RQ4: Does the advantage of ModernTCN generalize across multiple closed-loop routes?
- RQ5: Does the advantage remain under measurement noise and plant parameter perturbations?
- RQ6: Can the core inference and MPC computation satisfy a 10 ms control cycle?
- RQ7: Can offline perception metrics alone explain the closed-loop behavior of the causal ModernTCN ablation?

B. BASELINE CONTROLLERS AND ESTIMATORS

The compared methods are grouped into five categories. The first category is the zero-slope LPV-MPC baseline, denoted as LPV-MPC theta0. The second category is the IMU-based LPV-MPC baseline, which uses a simplified pitch-related slope estimate. The third category is the true-slope oracle LPV-MPC, which provides an upper-bound reference for scheduling quality. The fourth category includes learning-based scheduling perception methods: GRU, conventional TCN, and ModernTCN. The fifth category is the causal ModernTCN ablation.

C. CLOSED-LOOP ROUTES

The closed-loop evaluation includes one main showcase route and two additional route types. The three-route benchmark contains an industrial factory logistics route, a long up/down slope route, and a sharp turn transition route. These routes are used to test whether the method remains effective beyond a single trajectory.

fig3_closed_loop_routes.pdf

FIGURE 3. Closed-loop benchmark routes: (a) factory logistics showcase, (b) long up/down route, and (c) sharp turn transition route. Blue arrows indicate the travel direction.

D. EVALUATION METRICS

The evaluation metrics are divided into three groups. The first group measures tracking performance, including lateral-error RMSE, peak lateral error, heading-error RMSE, and global position RMSE. The second group measures control and constraint behavior, including control smoothness cost, force peak, yaw-rate command peak, constraint violation rate, and MPC solve time. The third group measures perception performance, including operating-condition accuracy, steering-direction accuracy, slope mean absolute error, and

confidence-related statistics.

This separation is important because non-AI and oracle baselines are not classifiers. Therefore, perception metrics should be interpreted only for learning-based perception models. The oracle baseline is used primarily as a tracking and control upper-bound reference, not as a perception classifier.

E. ROBUSTNESS PROTOCOL

Robustness is evaluated on two route types and three disturbance levels. The disturbance levels are denoted by $d = 0$, $d = 1$, and $d = 2$. In the public result files, $d = 0$ corresponds to no added disturbance, $d = 1$ uses noise scale 1 and process scale 0.35, and $d = 2$ uses noise scale 1.5 and process scale 0.7. The tested routes are the long up/down route and the sharp turn transition route.

F. REAL-TIME BENCHMARK PROTOCOL

The real-time benchmark separately reports the ONNXRuntime single-window inference time, MPC solve time, the computed ONNXRuntime-plus-MPC core cycle time, the MATLAB replay wrapper time, and the desktop Simulink wall time. This separation is necessary because MATLAB/Simulink desktop wrapper overhead should not be interpreted as embedded controller computation time.

VII. RESULTS AND DISCUSSION

A. MAIN CLOSED-LOOP COMPARISON WITH NON-AI, IMU, AND ORACLE BASELINES

Table 4 reports the main showcase-route comparison among the proposed ModernTCN-enhanced LPV-MPC, GRU, TCN, zero-slope LPV-MPC, IMU-based LPV-MPC, and true-slope oracle LPV-MPC. The zero-slope and IMU-based baselines show much larger lateral and global position errors than the learning-based and oracle methods. The true-slope oracle provides the best scheduling reference and achieves the smallest global position RMSE and very low control smoothness cost. ModernTCN achieves a lateral-error RMSE of 0.0246 m, which is close to the oracle result of 0.0236 m on this path, while also maintaining zero violation rate in the reported summary.

These results indicate that slope-related scheduling mismatch can dominate the closed-loop behavior on the showcase route. The nominal zero-slope and simplified IMU-based schedulers are unable to maintain acceptable path tracking over the full route. By contrast, ModernTCN substantially reduces the tracking error and gives the best overall performance among the learning-based controllers. The gap between ModernTCN and oracle remains visible in global position error and control smoothness, which indicates that better scheduling estimation and smoother scheduling conditioning remain meaningful future directions.

B. COMPARISON AMONG LEARNING-BASED CONTROLLERS

Among the learning-based controllers, ModernTCN achieves the lowest lateral-error RMSE and heading-error RMSE on the main showcase route. GRU produces a smaller scheduled-slope MAE than ModernTCN on this path, but its tracking errors are larger. TCN achieves the lowest raw slope MAE and the highest steering accuracy among the three learning-based methods on this path, but its control smoothness cost is much larger and it exhibits a nonzero violation rate. Therefore, the conclusion is not that ModernTCN is best in every single

metric. Rather, ModernTCN provides the most robust overall closed-loop tradeoff among the learning-based methods.

C. MULTI-ROUTE GENERALIZATION

Table 5 reports the aggregate results over three closed-loop paths. ModernTCN obtains the best mean overall rank and the best worst-case rank across all three paths. The oracle remains a strong upper-bound reference in global position error and control smoothness, but ModernTCN is the most consistent learning-based controller over the multi-route benchmark.

The multi-route result is important because it reduces the possibility that the main-route result is caused by a trajectory-specific tuning effect. The three routes cover factory logistics transportation, long up/down slope tracking, and sharp turning transitions. Across these cases, ModernTCN remains the best-ranked method according to the reported overall ranking.

D. ROBUSTNESS UNDER MEASUREMENT NOISE AND PLANT PERTURBATIONS

Table 6 summarizes the robustness benchmark under three disturbance levels. For each disturbance level, ModernTCN obtains the best mean overall rank among the learning-based controllers. At $d = 0$, $d = 1$, and $d = 2$, the mean overall rank of ModernTCN is 1.0, whereas GRU and TCN have worse mean or worst ranks. The result supports the relative robustness of ModernTCN under the designed disturbance cases.

The robustness benchmark currently uses two routes and the reported disturbance-level settings. Although the results are favorable to ModernTCN, broader statistical validation with additional random seeds and real sensor noise should be considered in future work.

E. OFFLINE PERCEPTION RESULTS

Table 7 reports selected offline perception metrics for ModernTCN and the causal ModernTCN ablation. The default ModernTCN achieves high main-condition and steering-direction accuracy on the test set. The causal ModernTCN seed 11 also achieves competitive offline metrics, including main-condition accuracy of 0.9783, steering-direction accuracy of 0.9006, transition-window steering accuracy of 0.7907, and slope MAE of 0.2507 degrees. Therefore, the causal model is not an obviously weak offline classifier.

Offline metrics are useful for interpreting model behavior, but they should not be treated as the primary evidence for a control paper. A perception model with similar offline accuracy can still produce different closed-loop behavior due to error propagation, scheduling discontinuities, and distribution shift induced by feedback.

F. CAUSAL MODERNTCN ABLATION AND OFFLINE-CLOSED-LOOP MISMATCH

Table 8 compares the default ModernTCN with the causal ModernTCN ablation on the main showcase path. Although the causal model has competitive offline metrics, its closed-loop lateral-error RMSE increases to 23.1318 m and its global

TABLE 4. Main Closed-Loop Results on the Factory Logistics Showcase Path

Controller	e_y RMSE	e_ψ RMSE	XY RMSE	$J_{\Delta u}$	Violation rate	θ_{sch} MAE
ModernTCN	0.0246	0.0438	0.8036	0.5991	0	0.9239
GRU	0.0623	0.1714	1.6641	7.0298	0	0.3415
TCN	0.0364	0.1061	1.3538	235.6737	5.2768×10^{-5}	1.7077
LPV-MPC theta0	26.2527	1.6517	32.0981	285.9290	0.1040	3.1339
LPV-MPC IMU theta	26.4718	1.6155	32.1210	285.8180	0.1040	2.9522
LPV-MPC oracle theta	0.0236	0.0315	0.5602	0.0609	0	0.0722

fig4_main_route_comparison.pdf

FIGURE 4. Main-route closed-loop comparison: (a) XY trajectory comparison, (b) lateral error over time, (c) heading error over time, and (d) scheduled slope comparison. The insets in (b) and (c) zoom in the learning-based controllers and the oracle baseline. The zero-slope and IMU-based baselines are omitted from the insets for readability. The black curve denotes the reference path in (a) and the true slope profile in (d).

TABLE 5. Aggregate Results Over Three Closed-Loop Routes

Controller	Paths	e_y RMSE mean	e_ψ RMSE mean	XY RMSE mean	$J_{\Delta u}$ mean	Violation mean	Overall mean	rank	Worst rank
ModernTCN	3	0.0319	0.0321	0.4889	13.3133	0	1.0000	1	1
LPV-MPC oracle theta	3	0.0347	0.0294	0.3951	0.4386	0	2.0000	2	2
GRU	3	0.0483	0.0851	0.9358	3.3528	0	3.3333	4	4
TCN	3	0.0489	0.0814	1.0695	313.9197	1.8×10^{-5}	3.6667	4	4
LPV-MPC IMU theta	3	8.8930	0.6669	12.1776	579.7648	0.0358	5.0000	5	5
LPV-MPC theta0	3	8.8276	0.6891	12.2527	644.0558	0.0357	6.0000	6	6

TABLE 6. Robustness Aggregate Results Under Disturbance Levels

Level	Controller	Cases	e_y RMSE mean	e_ψ RMSE mean	XY RMSE mean	$J_{\Delta u}$ mean	Rank mean	Worst rank
0	ModernTCN	2	0.0355	0.0262	0.3316	19.6704	1.0000	1
0	GRU	2	0.0413	0.0420	0.5716	1.5143	2.0000	2
0	TCN	2	0.0552	0.0691	0.9273	353.0426	3.0000	3
1	ModernTCN	2	0.0618	0.1185	1.2880	205.7775	1.0000	1
1	GRU	2	0.0765	0.1143	1.3874	307.3123	2.5000	3
1	TCN	2	0.0686	0.1104	1.4244	669.0753	2.5000	3
2	ModernTCN	2	0.0819	0.1519	1.6587	709.7593	1.0000	1
2	GRU	2	0.1170	0.1614	1.8842	292.1807	2.0000	2
2	TCN	2	0.0957	0.1657	1.8597	1101.8056	3.0000	3

TABLE 7. Selected Offline Perception Metrics

Model	Main acc.	Turn acc.	Transition turn acc.	θ MAE
ModernTCN seed 21	0.9807	0.9022	0.7757	0.2519
Causal ModernTCN seed 11	0.9783	0.9006	0.7907	0.2597

position RMSE increases to 27.4839 m. The result demonstrates a clear offline–closed-loop mismatch.

This result should not be interpreted as evidence that causal convolutions are generally unsuitable for control. Rather, it indicates that the particular causalization, training setting, and deployment interface used in this ablation did not yield robust closed-loop behavior. The result also supports the central methodological point of this paper: learning-enhanced MPC modules must be validated in the closed loop rather than judged only by offline classification or regression metrics.

G. REAL-TIME FEASIBILITY

Table 9 reports the real-time benchmark. The ONNXRuntime single-window inference has a p95 latency of 0.4507 ms, and the MPC solve time has a p95 latency of 0.0416 ms. The computed ONNXRuntime-plus-MPC core cycle has a p95 latency of 0.4923 ms, which is below the 10 ms control-cycle budget. By contrast, the MATLAB replay wrapper and desktop Simulink wall time exceed 10 ms. Therefore, the deployable core computation satisfies the timing benchmark, whereas the MATLAB/Simulink desktop wrapper time should not be interpreted as embedded controller computation time.

H. SAFETY, DEPLOYMENT, AND LIMITATIONS

The proposed method retains the MPC as the final constrained controller. The neural network provides scheduling information rather than direct control commands. This design preserves the interpretability of the model-based controller

and allows input and input-rate constraints to remain active. The deployment-side slope conditioning further limits abrupt scheduling changes.

Nevertheless, several limitations should be acknowledged. First, the current evidence is based on MATLAB/Simulink closed-loop simulation rather than hardware-in-the-loop or physical AGV experiments. Second, the IMU-based baseline is a simplified scheduler and should not be interpreted as representing all possible sensor-fusion estimators. Third, a formal closed-loop stability proof or robust invariant-set analysis is not currently provided. Fourth, the robustness benchmark uses the designed disturbance levels and reported cases; additional random seeds and real sensor-noise datasets would further strengthen the statistical conclusion. Fifth, large MATLAB data files, trained weights, path files, and closed-loop time-series logs are not included in the public repository and should be provided through an external archive for full reproducibility.

VIII. CONCLUSION

This paper presented a multi-task ModernTCN perception-enhanced LPV-MPC framework for path tracking of a diagonal dual-steer AGV on sloped factory routes. The proposed method estimates the operating condition, steering direction, and slope-related scheduling variable from a 128-step, 19-feature proprioceptive input window and uses the resulting estimates to enhance LPV-MPC scheduling. Compared with zero-slope and IMU-based LPV-MPC baselines, ModernTCN substantially improves closed-loop tracking on the main showcase route. Compared with GRU and conventional TCN, it provides the most robust overall closed-loop performance across the reported single-route, multi-route, and disturbance benchmarks. The true-slope oracle comparison indicates that ModernTCN approaches the oracle in lateral tracking on the main route, although a gap

TABLE 8. Closed-Loop Causal Ablation on the Showcase Path

Controller	e_y RMSE	e_ψ RMSE	XY RMSE	$J_{\Delta u}$	Main acc.	Turn acc.
ModernTCN	0.0246	0.0438	0.8036	0.5991	94.1375	81.3941
Causal ModernTCN	23.1318	1.2734	27.4839	268.8678	77.0988	62.2236
GRU	0.0623	0.1714	1.6641	7.0298	92.2168	80.0697
TCN	0.0364	0.1061	1.3538	235.6737	74.7137	86.0904

TABLE 9. Real-Time Benchmark Summary

Metric	Source	Mean ms	p50 ms	p95 ms	Max ms	p95 pass
ONNXRuntime single window	Python ONNXRuntime	0.3804	0.3659	0.4507	1.8839	Yes
MPC solve time	Closed-loop logs	0.0268	0.0243	0.0416	2.1766	Yes
ONNXRuntime plus MPC	Computed sum	0.4072	0.3902	0.4923	4.0605	Yes
MATLAB replay plus MPC	Computed sum	17.1613	15.2300	28.4304	44.6239	No
Simulink wall per step	Desktop simulation wall time	29.3024	29.3024	30.0940	30.0940	No

remains in global position error and control smoothness. The causal ModernTCN ablation further shows that offline perception metrics alone are insufficient to predict closed-loop control robustness. Finally, the timing benchmark indicates that the core ONNXRuntime inference plus MPC solving pipeline has sufficient margin for a 10 ms control cycle, while MATLAB/Simulink desktop wrapper overhead should be treated separately. Future work will focus on hardware-in-the-loop and real-vehicle validation, stronger statistical robustness evaluation, confidence-triggered fallback scheduling, and formal analysis of scheduling-error effects on constrained closed-loop behavior.

...